GitHub: https://github.com/NssGourav/coffee-and-quiet

Coffee and Quiet Technical Design

Prepared by NssGourav · 9 November 2026

Live application: https://coffee-and-quiet.vercel.app

A public review deployment and a reproducible local setup are provided. Payments and sending are simulated; AI inference is real. Use fictional data only.

1. Product and selected scope

Coffee & Quiet implements Problem 1: a Chikkamagaluru coffee-estate resort's direct-booking website and AI enquiry inbox. The supplied property has 64 rooms and 12 villas. Guests need to see availability and complete a direct booking; reservations staff need dependable website stock and faster enquiry triage.

I chose a narrow end-to-end slice: room discovery, a three-step booking flow with simulated payment, rate/allotment administration, and a saved enquiry inbox with real AI classification and drafting. The hardest correctness requirement is preventing the website from selling more than its allotted stock on any occupied night. Extra visual polish supports that workflow; it does not replace it.

Delivered core

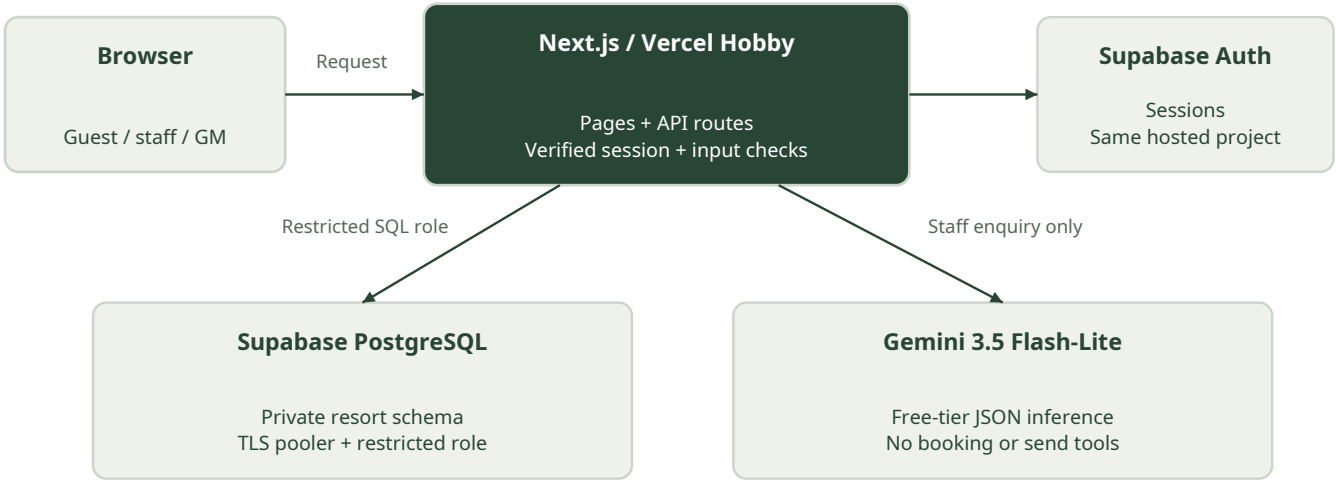
Requirement	Implemented behavior
Rooms with availability and tax-inclusive prices	Three seeded room types; multi-night availability and per-night price breakdown
Booking and confirmation	Sign-in, one-unit hold, ten-minute expiry, simulated success/failure, saved confirmation
Staff rates and allotment	Per-night edits, optimistic version checks, committed-stock protection and audit
AI enquiry inbox	Saved original text, classification, extraction, team routing, draft, correction and manual fallback
Human control	GM-visible complaint/unsorted lanes; staff explicitly review and simulate sending
Reviewer access	Four synthetic guest/staff accounts, realistic inventory and supplied enquiry examples

The eight screen groups are discovery; room detail; details/sign-in; review/payment; confirmation and saved bookings; rate/allotment desk; enquiry intake/inbox; and enquiry review/quote. Routes can be revisited directly; the booking stepper itself has three steps.

Stack and tradeoffs

Next.js 16.4.5, React 19.2 and TypeScript 5.9 form one application. PostgreSQL 17 owns business rules and durable data. Supabase Auth supplies password sessions. Zod validates inputs and model output; node-postgres uses parameterized SQL. Custom CSS and a route-specific Three.js 0.187 scene provide the interface. The live app uses Gemini 3.5 Flash-Lite on a confirmed free-tier project; local setup uses Ollama/Qwen3 4B. Tests use Node/tsx, PostgreSQL, HTTP and Playwright.

2. Architecture and boundaries



Local alternative: Next.js + native PostgreSQL + Supabase Auth + Ollama.

The browser sends requests to Next.js. Server pages read data and API routes validate mutations. Supabase verifies the session; the server then sets the verified user's ID inside a database transaction. The application never accepts a client-supplied actor ID. A restricted SQL role can read through row-level policies and call approved functions; it cannot directly edit business tables.

The live application runs on Vercel Hobby with one free Supabase project for both Auth and the private application schema. The backend uses the Supabase transaction pooler over certificate-verified TLS and one connection per function instance. Transaction-local identity and unnamed parameterized queries work across pooled connections. Owner/service keys are excluded from the app runtime. Public registration is disabled.

Local development retains two PostgreSQL installations: a dedicated native cluster for application data and Supabase's containerized database for Auth. They have different responsibilities; application rows are not duplicated between them. profiles.id copies the verified Auth user UUID; there is no cross-database foreign key. This local separation makes isolated database tests straightforward but adds setup weight.

AI is called only by the staff enquiry route. The original enquiry is saved first, model calls run outside a database transaction, and the result is committed only if its revision is still current. The model has no booking, payment or outbound-message tools.

Why this design

- One full-stack application keeps permissions, validation and deployment concerns in one codebase. Separate frontend and API services would add coordination without helping this slice.
- PostgreSQL transactions and shared inventory-row locks make the overbooking rule enforceable across competing requests. An in-memory counter would fail across processes and restarts.
- SQL functions centralize all stock-changing writes. Security-definer functions have fixed search paths and explicit grants; their owner privileges are not exposed as runtime credentials.
- Hosted inference makes the live app usable without an always-on personal computer. Only fictional enquiry data is intended; the form and credits disclose AI processing. Free quotas and provider outages can require manual review. Local Ollama remains the private/offline option after initial downloads.
- No queue, vector database or retrieval framework is required for classification and category-based drafting. Deterministic pricing is read from the rate table.

Both deployments share the same business logic and SQL migrations. AI_PROVIDER explicitly selects the model service; there is no silent paid fallback. The deployed application never calls a loopback Ollama endpoint. Free service limits can interrupt availability, so the local setup remains a reviewer fallback.

3. Data model and access

Entity	Key data and purpose
profiles	Auth UUID, display name, guest/staff role and GM flag
room_types	Room ID, capacity, physical key count, descriptive content and image
inventory	Composite key (room_type, night), website allotment, base rate in paise, version
bookings	Guest, room, date interval, status, expiry, idempotency key, total and policy snapshot
booking_nights	Composite key (booking_id, night), accepted base, GST, tax, rule ID and rate version
payment_events	Unique event ID, booking, simulated outcome and recorded result
inventory_audit	Actor, room/night, old and new rates/allotments, timestamp
enquiries	Guest, immutable original, working message, category, owner team, GM alert, extraction, draft, state and version
enquiry_audit	Actor, action, prior revision/message/category and final simulated-send text

A profile owns many bookings and enquiries. A room type has many inventory nights and bookings. A booking has many nightly snapshots and payment events. An enquiry has many audit entries. No room-level allocation or channel-manager inventory is represented.

Guests can read only their own bookings/enquiries. Staff can read all bookings and work the inbox, quotes and inventory. GM is a staff account with an additional flag; the complaint and unsorted lanes are visible to staff, including the GM. There is no separate GM-only authorization boundary or external alert service.

Dates are calendar dates in the resort's India context, with checkout excluded from occupied nights. Money is integer paise. Booking prices are copied into nightly snapshots; later inventory edits do not recalculate accepted bookings. The implemented accommodation bands are 5% through INR 7,500 per unit/night and 18% above, with boundary/rounding tests. This records the implementation; tax invoicing and legal applicability require separate production review. See docs/tax-sources.md for the dated source assessment.

4. Booking and allotment correctness



For every room type r and occupied night n:

confirmed(r,n) + unexpired_holds(r,n) <= website_allotment(r,n)

Booking flow

1. Search validates dates and occupancy. Availability is computed per occupied night; the complete stay must be available.
2. The room/detail page shows nightly amounts, GST and the total. The guest submits the expected total and ordered inventory version vector.
3. The server verifies the session and input. A guest/request-key advisory lock serializes retries. The same key and same payload return the earlier booking; changed payload is rejected.
4. Within READ COMMITTED, the hold function locks every inventory row for the room/date range in ascending date order. A missing night rejects the whole stay.
5. After lock acquisition, it captures the database clock and recounts commitments using a fresh statement snapshot. It validates capacity, stock, total and every quoted version.
6. A successful transaction inserts one ten-minute hold and all nightly snapshots atomically. A failed transaction inserts none.
7. Simulated payment locks the booking and the same inventory rows. It rechecks expiry after waiting, records the unique event and confirms or releases the hold. An expired booking cannot be resurrected by a late success.
8. The saved confirmation and My bookings read the persisted result. A lost browser response does not erase an accepted booking.

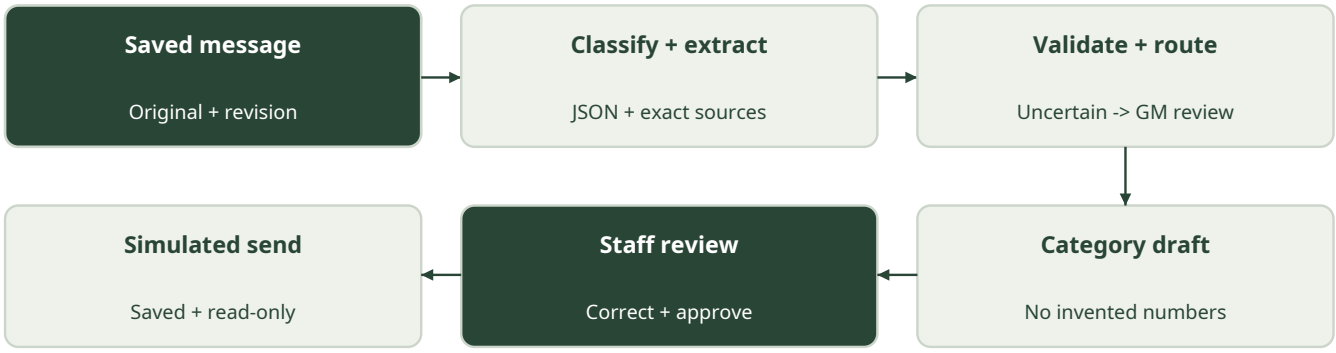
Staff edits and races

Allotment edits lock the same inventory row, reject stale versions and enforce live commitments <= new allotment <= physical key count. Setting zero closes an uncommitted night. Successful edits increment the version and record an audit entry.

The nightly version vector matters even if two rate changes cancel out and leave the total unchanged. A changed quote must be reviewed again. Expired holds are excluded by the clock predicate, so stock recovery does not depend on a scheduled cleanup task.

Observed tests include 12 competing clients for the final unit yielding exactly one hold, overlapping date ranges, rollback on partial availability, duplicate request/payment keys, rate edits versus accepted snapshots, and a payment that becomes expired while waiting for a lock. These support the implemented invariant, not an unlimited-load or zero-production-incident claim.

5. AI enquiry flow



The seeded examples include the supplied wedding, villa-availability and offsite enquiries, plus a complaint and an unclear request. Guests can also submit new messages through the web form. Staff explicitly start AI; email/WhatsApp intake and background workers are deferred.

Actual model calls

1. Save the original message and assign a version before any model work. An explicit complaint flag immediately sets a sticky GM alert.
2. Call the configured model to classify room, wedding, offsite, complaint or unsorted, returning category and confidence as JSON.
3. For room/wedding/offsite categories, make a second call for dates, guest counts and budget. Values must be exact contiguous source substrings. Do not infer a year or fill absent fields.
4. Validate strict JSON schemas and source matches. Confidence below 0.8, missing evidence or a conservative suspicious-instruction check routes to unsorted. The threshold is an assumption and is not calibrated.
5. Apply deterministic team routing: room to Reservations; wedding/offsite to Event sales; complaint to GM; unsorted to GM review. Explicit staff routing is preserved. Complaint alerts stay sticky.
6. Make a draft call using the validated category only. Request a brief clarification reply without prices, dates or unsupported availability. Reject malformed output and drafts containing numbers/currency symbols.
7. Save the result with a revision check. If staff edited while inference ran, the stale result cannot overwrite the newer revision. Failures retain the original and offer manual review/retry.
8. Staff inspect/edit the draft, optionally append an authoritative quote, approve and simulate send. The saved sent record becomes read-only and an audit entry records the action. No external message leaves the app.

Typical processing uses three sequential calls. Gemini calls time out after 25 seconds each; the hosted route allows 90 seconds. Local Ollama calls allow 120 seconds each. Failures retain manual review.

Quotes and safety limits

The staff-only quote tool uses confirmed dates and occupancy to read current nightly rates/availability and calculate taxes. Staff append its timestamped indicative quote. It neither invents event tariffs nor reserves stock; booking rechecks prices and availability.

Drafts ask for clarification; they are not personalized fact-grounded responses. Source matching proves copying, not comprehension. Evidence shows the first 500 characters; the full original remains visible. The injection heuristic is incomplete. Human review remains necessary.

Approval is an explicit UI control; the server enforces staff identity, current revision and draft bounds. There is no separate server-persisted approval receipt or send-time quote revalidation. Duplicate send attempts are rejected rather than replayed as an idempotent success. Enquiry intake is rate-limited to 20 per user/hour but has no idempotency key. These are limits to address before real outbound integration.

6. UI and experience decisions

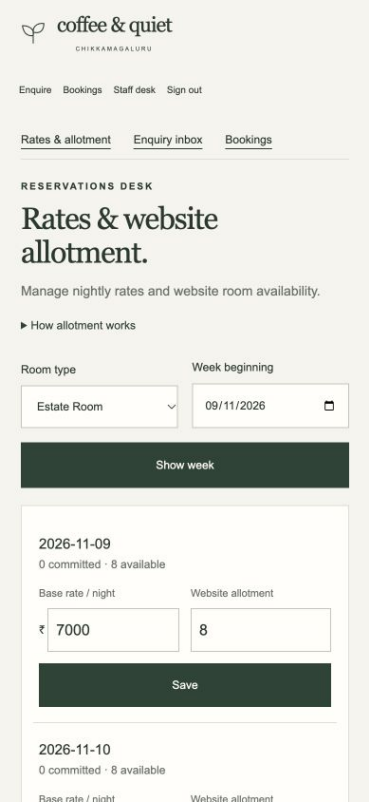
The resort's quiet character is expressed through warm off-white, restrained forest green, serif headings and simple forms. Photography is large on discovery and room pages; rate totals and next actions remain easy to locate. The brand, room descriptions and photos are explicitly illustrative, with credits linked from the footer.

The booking stepper is choose stay, details/sign-in, then review/payment. Confirmation is the result, not another decision step. Price changes, expiry and sold-out outcomes provide a recovery action. Busy states prevent accidental repeated taps; retained inputs and saved bookings help recover from interrupted requests.

Staff pages use denser spacing and smaller headings than discovery. Rates and allotment are editable per night with visible commitments; the longer editing guidance lives in accessible expandable help. Inbox filters separate attention, GM/unsorted and sent work. The editor places original text and working reply together, with errors and stale-edit feedback close to the action.

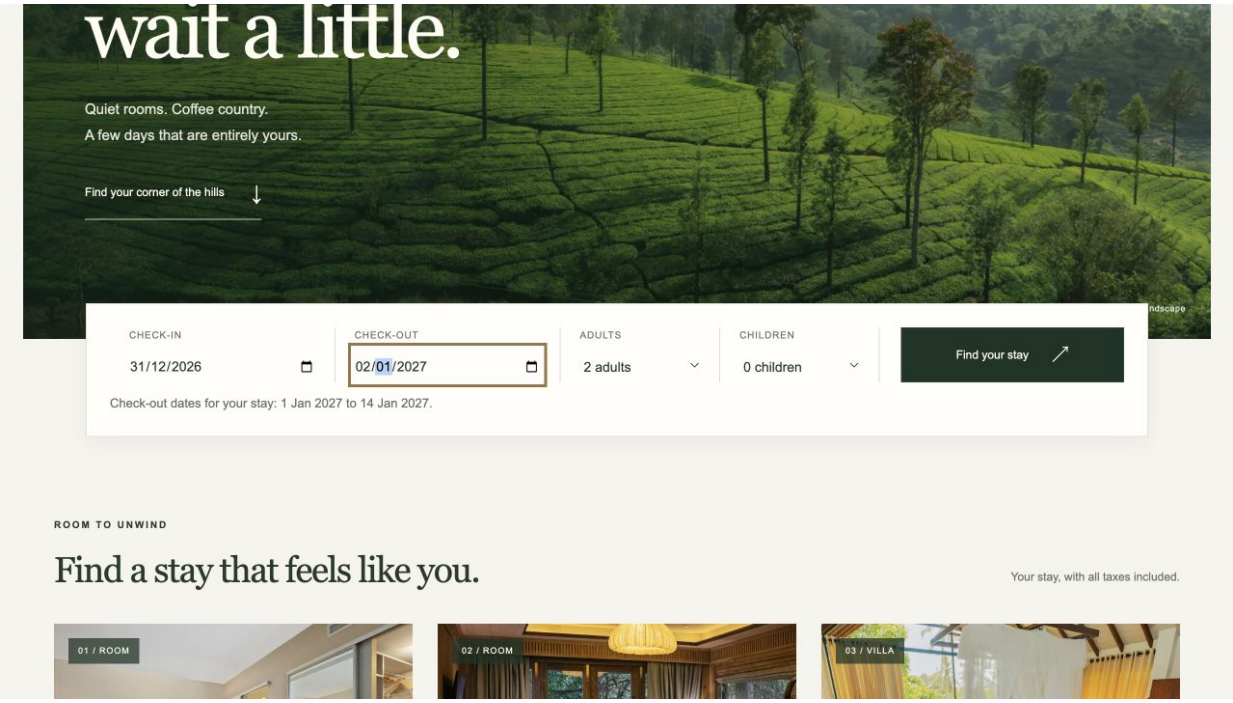
The sign-in page lazily loads a procedural Three.js landscape. It uses no downloaded model/texture assets, caps render resolution, pauses when offscreen/hidden and disposes resources on unmount. A static illustration remains available if loading/WebGL fails. Reduced-motion settings show a static scene; the form renders independently. This is optional presentation polish, not evidence of faster conversion.

The search form updates checkout guidance when check-in changes, bounds the calendar to 1-14 nights, and keeps server validation authoritative. Clear empty/error messages help guests recover. Mobile layouts stack forms and staff rows, preserve labels and retain 46px inventory controls. A skip link, visible focus styles, semantic labels, alert/status messages and keyboard-operable disclosure help support accessibility. Native dates and system fonts reduce dependencies. Room photos are optimized WebP with responsive Next.js Image sizing. Slow-network performance and comprehensive screen-reader compliance have not been measured.

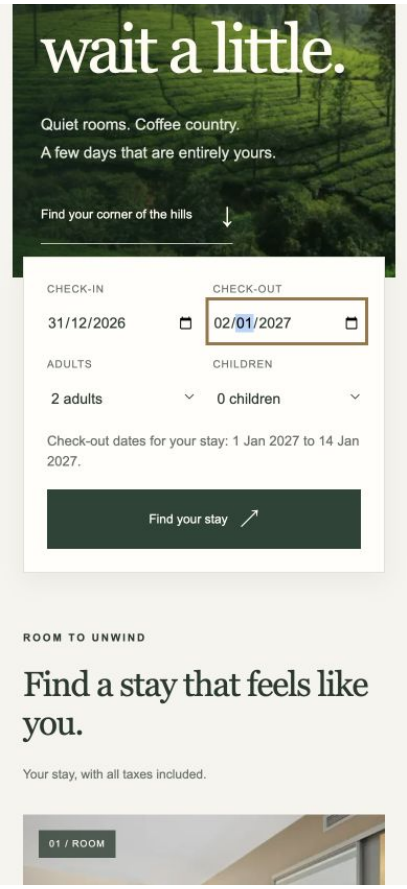


Responsive check: at 390px, inventory becomes labelled cards and the document fits the viewport. Screenshot from the fresh production build.

Interface evidence - stay search

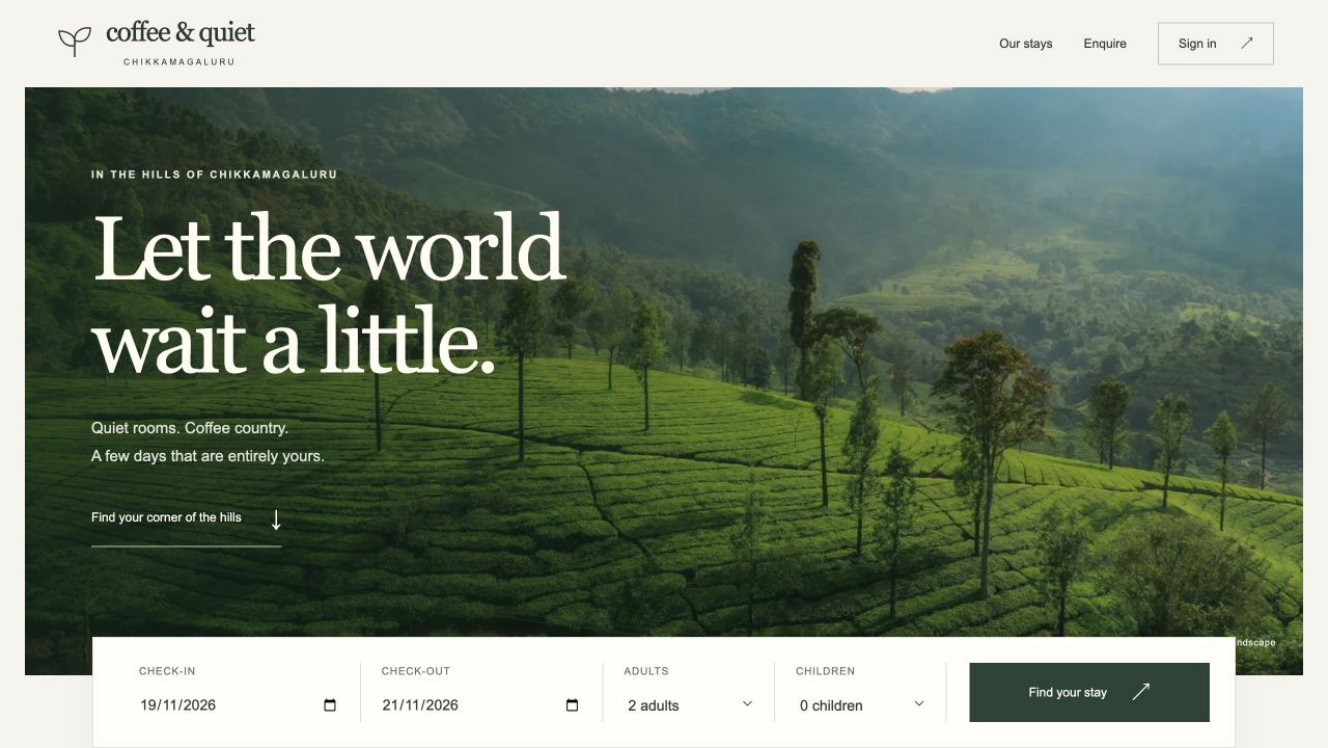


Final search update: a check-in change updates the allowed checkout range across New Year. Valid searches return the matching room cards; server validation independently enforces the same limits.

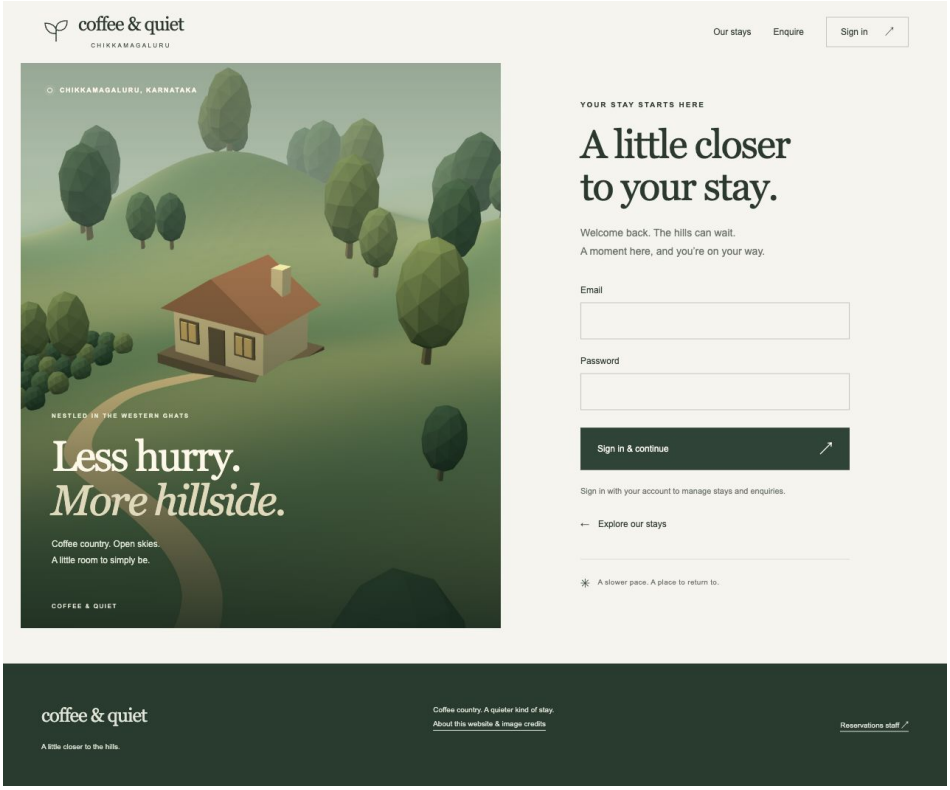


At 390px, date guidance sits above the full-width search button with no horizontal document overflow. Both captures are from the final production build.

Interface evidence - guest journey

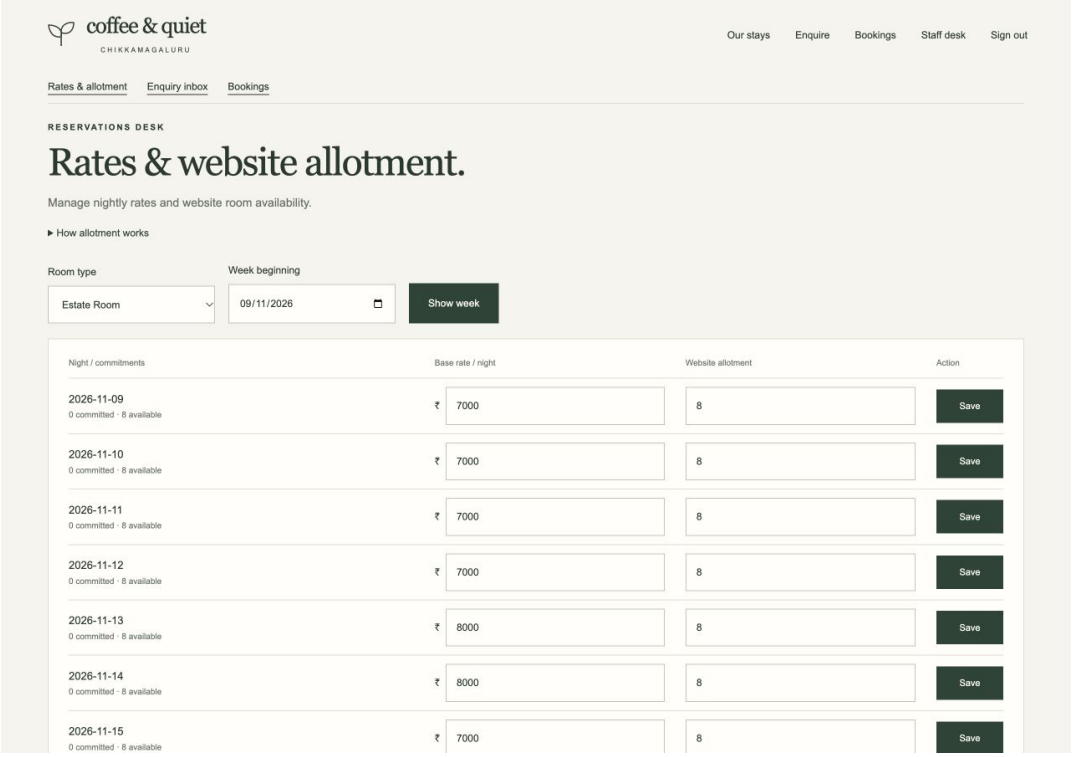


Live discovery: landscape-led identity and a visible date/occupancy search. Captured from the hosted application, 9 November 2026.

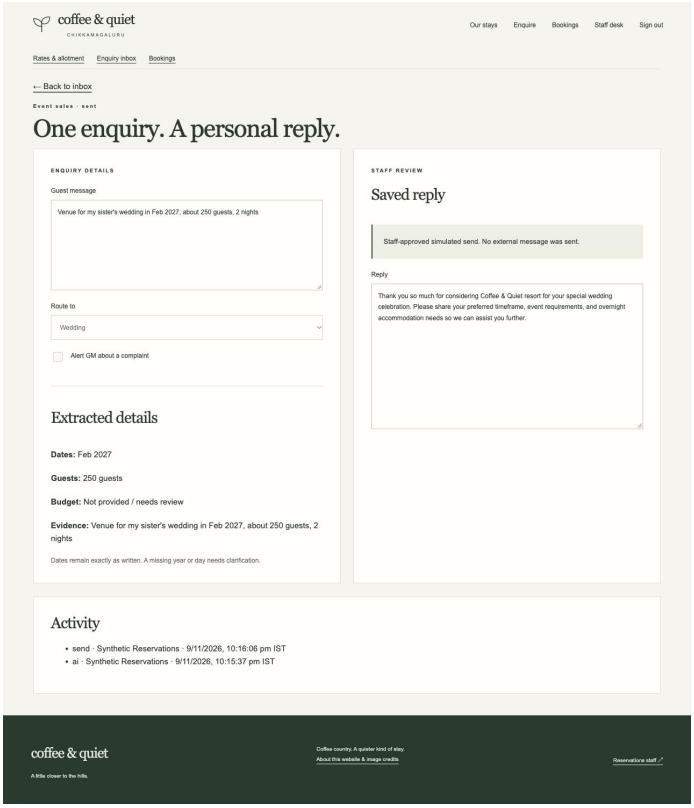


Sign-in: a lazily loaded procedural Three.js scene sits beside the independent sign-in form. The static fallback and reduced-motion behavior keep the form usable.

Interface evidence - reservations



Inventory: compact navigation, live commitments, nightly price/allotment inputs and one save action per row. Further nights continue below this viewport.



Live AI flow: Gemini classified and extracted the supplied wedding enquiry. After explicit staff approval, the simulated reply was saved and made read-only. No external message was sent.

7. Verification and security

Verification is recorded in docs/evidence.md, with real AI outputs in docs/ai-evaluation-final.json and docs/ai-evaluation-hosted.json. Unit/model-transport tests use stubs for deterministic failure checks; separate AI evaluations call the real local and hosted services.

Layer	Evidence and scope
Unit	14 checks for calendar dates, tariff boundaries, expiry, validation, source extraction, uncertainty and malformed/unavailable model responses
PostgreSQL	22 checks including competing holds, partial stays, expiry-after-lock, duplicate events, stale edits, guest isolation and enquiry revision/send races
HTTP	Authenticated booking and enquiry checks against Next.js, real Supabase Auth and PostgreSQL
Real model	Local: 7/7 cases. Hosted: 6/7 on the first run; one provider failure passed on retry. Not a general accuracy benchmark
UI	Guest booking, staff inventory and AI editor journeys; mobile/desktop layout, keyboard help and sign-in error/success checks
Build	Local hosted-configuration build and Vercel production build passed, including TypeScript
Live app	Public access, sign-in, hold retries, confirmation, guest isolation, staff inventory and real AI through approved simulated send passed
Reproduction	Separate public clone, locked install and fresh local data on the development Mac; temporary network-port relocation avoids disturbing the working app

Security controls include server-verified identities, bounded JSON bodies, schema validation, same-origin mutation checks, parameterized SQL, row-level access policies and restricted function grants. Owner and Auth service-role credentials are used only for provisioning. The app rejects an owner/superuser/bypass-RLS runtime connection. Client keys are public Auth keys, not database secrets. The private application schema is not exposed through Supabase's Data API.

Secrets and model weights are ignored; generated local files have restricted permissions. The publication check scanned working source and historical Git blobs for known local credential values and common token/private-key patterns. That focused check and a clean dependency advisory result are not a comprehensive security audit.

8. Assumptions and deliberate cuts

Client facts are the location, coffee-estate setting, 64 rooms plus 12 villas, guest/reservations/GM users, direct-booking and enquiry problem, required core, simulated integrations and website-only allotment. The following are implementation choices, not supplied property facts.

ID	Labelled assumption
A1	Fictional Coffee & Quiet brand, proposed copy and licensed illustrative photos
A2	Physical split: 40 Estate Rooms, 24 Terrace Rooms, 12 Garden Villas; capacities 3, 3 and 4
A3	Base rates INR 7,000 / 9,500 / 14,500; Friday/Saturday add INR 1,000; initial allotments 8 / 5 / 3
A4	One unit per booking, room-only plan, 1-14 nights, check-in within one year, initial 121 seeded nights
A5	Ten-minute holds; checkout-exclusive occupied nights; integer-paise, per-night tax rounding
A6	Four provisioned synthetic accounts; no public registration/recovery; shared staff visibility including GM
A7	Staff-triggered web enquiries; team-level ownership rather than named employee assignment
A8	Local Qwen3 4B / hosted Gemini 3.5 Flash-Lite; separate classify/extract/draft calls; 0.8 uncertainty threshold and original-text extraction
A9	No guessed years, event package tariffs or automatic promises; staff confirms dates and appends checked room quotes
A10	Local native PostgreSQL plus Supabase Auth; hosted Vercel Hobby plus one free Supabase project and free Gemini inference

Deferred optional depth: promo codes, richer rate plans, structured event form, daily booking export, forwarded-email/WhatsApp intake simulation and bulk inventory editing. Blackout behavior is represented by zero allotment where commitments permit it, rather than a separate blackout entity.

No channel-manager replacement, loyalty or restaurant booking is attempted. No real payments/messages, cancellation/refund integration, passport collection, GST invoices or privacy-compliance certification are claimed.

9. Next steps and business interpretation

The live review deployment is an assessment environment with shared synthetic accounts. It has no uptime guarantee: Supabase can pause free projects after inactivity, and the model can reject requests when quotas are exhausted. Reviewer credentials are in the submission copy of deployment.md; operational credentials are excluded from Git. Keep the repository public through review and monitor the free services during that period.

Before real operations, add revision-bound approval receipts, send-time quote checks and an idempotent outbound queue; calibrate uncertainty on a larger labelled set; evaluate latency and low-bandwidth performance; complete registration/recovery, retention and deletion procedures; and commission accessibility, privacy and tax reviews. Add optional intake/export features only after the core remains reliable.

The brief's financial scenario is a target, not an achieved outcome. Using its unrounded inputs gives about INR 33.33 lakh annual commission savings plus INR 72 lakh event contribution, around INR 1.053 crore. Moving 15% of OTA nights increases direct share by 10.2 percentage points, from 12% to 22.2%, which differs from the stated 20% target. No production revenue, response-time or occupancy improvement has been measured.

The project was built individually with AI assistance for planning, implementation, review and documentation. Commit history preserves actual milestones. This document describes the implemented design and its limits so the choices can be explained in an interview.

References

- Authoritative assessment and selected Problem 1 supplied by the evaluator.
- Repository: https://github.com/NssGourav/coffee-and-quiet
- Application decisions and assumptions: docs/decisions.md and docs/assumptions.md.
- Dated verification and source notes: docs/evidence.md, docs/tax-sources.md and docs/assets.md.
- Supabase local setup: https://supabase.com/docs/guides/local-development
- Ollama model/API references: https://ollama.com/library/qwen3:4b and https://docs.ollama.com/api/chat
- Three.js rendering reference: https://threejs.org/docs/pages/WebGLRenderer
- Hosted service references: https://supabase.com/docs/guides/database/connecting-to-postgres and https://vercel.com/docs/functions/limitations
- Gemini model and free-tier pricing: https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash-lite and https://ai.google.dev/gemini-api/docs/pricing